

A PROJECT REPORT ON

CredentialChain

Blockchain-Based Academic Credential System
with Verifiable Credentials

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science in Information Technology

Submitted by

Cecil Baraik

Enrolment / Roll No.: 2025-BSCIT/004A

B.Sc. Information Technology · Batch 2025

Under the guidance of

(Project Guide)

Amity University Jharkhand

Ranchi, Jharkhand

2026

Certificate

This is to certify that the project report entitled "**CredentialChain - Blockchain-Based Academic Credential System with Verifiable Credentials**" is a bona fide record of the work carried out by **Cecil Baraik** (Roll No. 2025-BSCIT/004A), student of B.Sc. Information Technology, Amity University Jharkhand, under my supervision and guidance, in partial fulfilment of the requirements for the award of the degree.

Project Guide

Head of Department

Date: _____ Place: Ranchi

Declaration

I, **Cecil Baraik**, hereby declare that the project report entitled "**CredentialChain - Blockchain-Based Academic Credential System with Verifiable Credentials**" submitted to Amity University Jharkhand is a record of my own work. To the best of my knowledge, it has not been submitted elsewhere for the award of any other degree or diploma.

Cecil Baraik
Roll No. 2025-BSCIT/004A

Acknowledgement

I would like to express my sincere gratitude to my project guide for the guidance, encouragement and valuable suggestions given throughout this project. I am thankful to the faculty of the Department of Information Technology, Amity University Jharkhand, for providing the knowledge and resources needed to complete this work.

I also thank my family and friends for their constant support and motivation during the development of this project.

Cecil Baraik

Abstract

Fake and forged academic certificates are a serious problem for employers and universities. Verifying a certificate today usually means contacting the issuing institution by email or phone, which is slow, manual and easy to abuse.

This project presents **CredentialChain**, a web application that issues tamper-evident digital academic credentials and lets anyone verify them in seconds. When an institution issues a credential, the system computes a SHA-256 fingerprint of the record, signs it with the institution's ECDSA (P-256) private key and records only the fingerprint on a custom proof-of-work blockchain written in Java. The full personal record stays in a conventional database, protecting privacy. Institutions' public keys are themselves anchored on the chain, so a compromised database cannot introduce a fake issuer key.

Verification runs six independent checks: record existence, blockchain anchoring, data-to-hash match, issuer signature, revocation status and full-ledger integrity. Credentials can be verified by credential ID, by scanning the QR code on a printable PDF certificate, or by uploading the signed JSON document. The system supports revocation without deleting history, bulk issuing from CSV files, role-based login with salted PBKDF2 password hashing and signed JWT tokens, and both H2 and MySQL databases.

Testing shows that editing a downloaded certificate, changing a grade directly in the database, or altering a mined block are all detected immediately. The project demonstrates that blockchain concepts such as hash chaining, Merkle trees, proof-of-work and digital signatures can make academic credential verification fast, trustworthy and independent of the issuer being online.

Keywords: blockchain, verifiable credentials, SHA-256, ECDSA, proof-of-work, Merkle tree, Spring Boot, React

Table of Contents

1. Introduction	8
1.1 Background	8
1.2 Problem statement	8
1.3 Objectives	8
1.4 Scope	8
2. Literature Review and Existing Systems	10
2.1 Traditional verification	10
2.2 Blockchain foundations	10
2.3 Blockchain-based credentials	10
2.4 Comparison	10
3. System Analysis	12
3.1 Users and roles	12
3.2 Functional requirements	12
3.3 Non-functional requirements	12
3.4 Feasibility study	13
3.5 Hardware and software requirements	13
4. System Design	14
4.1 Architecture	14
4.2 Credential issuance flow	14
4.3 Blockchain structure	15
4.4 Verification flow	15
4.5 Database design	15
4.6 Transaction types	16
4.7 Module description	16
5. Implementation	17
5.1 Technology stack	17
5.2 Hashing a credential	17
5.3 Proof-of-work mining	17
5.4 Merkle root	17
5.5 Digital signatures	18
5.6 Chain validation	18
5.7 Authentication	18
5.8 Main REST endpoints	18

6. Results and Screenshots	19
7. Testing	26
7.1 Testing approach	26
7.2 Test cases	26
8. Security Analysis	27
9. Limitations and Future Scope	28
9.1 Limitations	28
9.2 Future scope	28
10. Conclusion	28
References	28

1. Introduction

1.1 Background

Academic certificates are among the most important documents in a person's career. Employers, universities and governments rely on them to make hiring and admission decisions. However, paper certificates and even scanned PDFs are easy to forge with modern editing tools, and confirming their authenticity usually requires contacting the issuing institution directly.

Blockchain technology offers a different approach. A blockchain is an append-only ledger in which every block contains the cryptographic hash of the previous block. Any change to past data breaks this chain of hashes and is immediately visible. Combined with digital signatures, this makes it possible to publish a permanent, verifiable proof that a specific institution issued a specific credential, without publishing the private data itself.

1.2 Problem statement

- Fake certificates are easy to create and hard to detect by looking at them.
- Manual verification through the issuing institution is slow, costly and depends on the institution responding.
- Centralised databases can be altered by insiders without leaving a trace.
- There is no simple way for a student to share a credential that anyone can verify independently.

1.3 Objectives

- Build a system in which authorised institutions issue digitally signed academic credentials.
- Record a tamper-evident fingerprint of each credential on a blockchain implemented from first principles.
- Allow anyone to verify a credential in seconds by ID, QR code or uploaded file, without logging in.
- Support revocation of credentials while keeping a complete, auditable history.
- Detect and clearly report forged documents, database tampering and blockchain tampering.
- Protect the system with secure login, role-based access and safe password storage.

1.4 Scope

The project covers credential issuance, verification, revocation, bulk issuance, printable PDF certificates, a blockchain explorer and an attack-simulation lab for demonstration. It runs as a single-node permissioned blockchain suitable for one university or a small consortium. Multi-node consensus and integration with public

blockchains are outside the current scope and are discussed as future work.

2. Literature Review and Existing Systems

2.1 Traditional verification

In the traditional process, a verifier contacts the registrar of the issuing institution, who checks internal records and replies. Third-party background-verification agencies automate parts of this but still depend on the institution's records and add cost and delay. Neither approach protects against an insider who alters the institution's own database.

2.2 Blockchain foundations

Bitcoin (Nakamoto, 2008) introduced a public ledger secured by hash-linked blocks and proof-of-work, showing that a shared record can be made practically impossible to rewrite. Merkle trees (Merkle, 1987) allow one hash to commit to many transactions. These two ideas are the core of the blockchain implemented in this project.

2.3 Blockchain-based credentials

Blockcerts, an open standard started by the MIT Media Lab and Learning Machine, anchors hashes of academic certificates on public blockchains so that recipients can share and verify them independently. The *W3C Verifiable Credentials Data Model* defines a standard structure for digitally signed credentials containing a subject, an issuer and a cryptographic proof.

CredentialChain follows the same principles: only a fingerprint is anchored on-chain, the issuer signs it, and the downloadable credential document has a *credential* section and a *proof* section loosely modelled on the W3C data model. Unlike public-chain solutions, it runs on a self-built permissioned chain, which avoids transaction fees and makes every component visible for study.

2.4 Comparison

Feature	Manual verification	Central database	CredentialChain
Verification time	Days	Minutes (if access given)	Seconds
Works without issuer responding	No	No	Yes
Detects insider edits	No	No	Yes (hash mismatch)
Detects forged documents	Partly	Partly	Yes
Revocation with history	Manual	Overwrites	Yes (REVOKE transaction)
Personal data on public ledger	-	-	No (hash only)

Table 1: Comparison of verification approaches

3. System Analysis

3.1 Users and roles

Role	Description	Main actions
Administrator	Operates the platform	Register institutions, create staff logins, use the attack lab
College staff (Issuer)	Registrar of an institution	Issue credentials (single or CSV), revoke, download PDFs
Verifier	Employer or any third party	Verify by ID, QR code or JSON file – no login needed
Student	Credential holder	Find credentials by roll number, share link / PDF / JSON

Table 2: Users of the system

3.2 Functional requirements

- FR1 – The administrator can register an institution; the system generates its ECDSA key pair and anchors the public key on-chain.
- FR2 – College staff sign in with email and password.
- FR3 – Staff issue credentials individually or in bulk from a CSV file; future issue dates are rejected.
- FR4 – Each credential is hashed (SHA-256), signed (ECDSA) and recorded in a pending transaction.
- FR5 – Pending transactions are mined into blocks by proof-of-work, manually or automatically every 30 seconds.
- FR6 – Anyone can verify a credential by ID, QR code or uploaded JSON document and see every check performed.
- FR7 – The original issuer can revoke a credential with a reason.
- FR8 – Credentials can be downloaded as a signed JSON document and a printable PDF with a QR code.
- FR9 – The whole chain can be validated on demand; tampering is reported block by block.
- FR10 – The dashboard shows statistics and charts of blocks, transactions and mining effort.

3.3 Non-functional requirements

- **Security:** salted password hashing, signed expiring tokens, role checks, brute-force lockout.
- **Integrity:** any change to issued data must be detectable.
- **Privacy:** no personal data on the blockchain.
- **Usability:** verification in a few clicks; clear results in plain language; works on mobile screens.

- **Portability:** runs on Windows, Linux or macOS with Java and Node.js; H2 or MySQL database.
- **Maintainability:** layered code (controllers, services, repositories) and automated tests.

3.4 Feasibility study

Technical: all components use mature open-source technologies (Java, Spring Boot, React) and standard cryptography built into the Java platform. **Economic:** no licence or blockchain transaction fees are required; the system runs on an ordinary PC. **Operational:** staff need only a browser; verifiers need no account and no special software.

3.5 Hardware and software requirements

Item	Requirement
Processor / RAM	Any modern dual-core CPU, 4 GB RAM or more
Operating system	Windows 10/11, Linux or macOS
Backend	Java 17 or newer (developed on Java 21), Spring Boot 3.3, Maven
Frontend	Node.js 18 or newer, React 18, Vite 5
Database	H2 (embedded, default) or MySQL 8
Tools	IntelliJ IDEA, Git and GitHub, a modern web browser

Table 3: Hardware and software requirements

4. System Design

4.1 Architecture

The system follows a three-tier architecture. A React single-page application talks to a Spring Boot REST API, which contains the authentication, credential, blockchain and cryptography modules and persists data through JPA to H2 or MySQL.

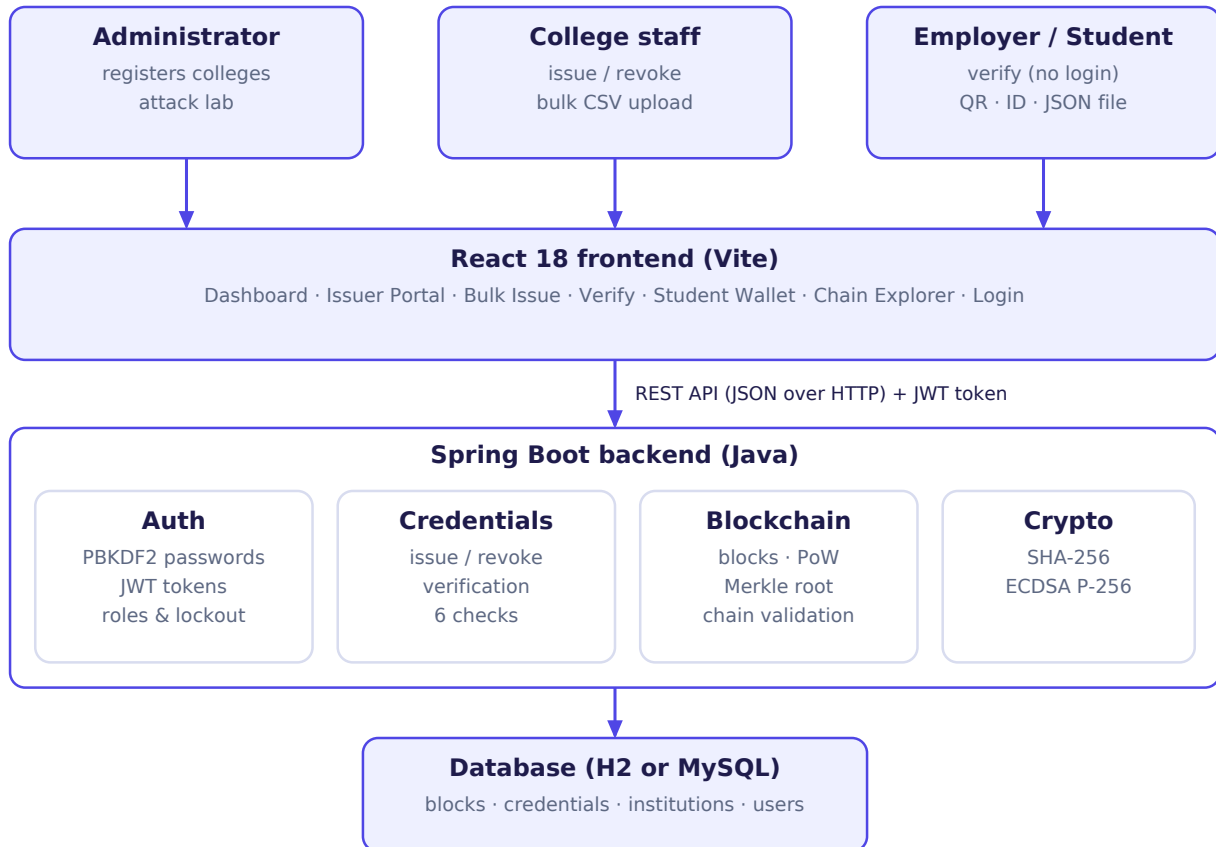


Figure 1: System architecture

4.2 Credential issuance flow

When staff submit the form, the server builds a canonical text representation of the credential, hashes it with SHA-256 and signs the hash with the institution's private key. An ISSUE transaction containing only the hash, the issuer ID and the signature is added to the pending pool. The complete record is stored in the *credentials* table.

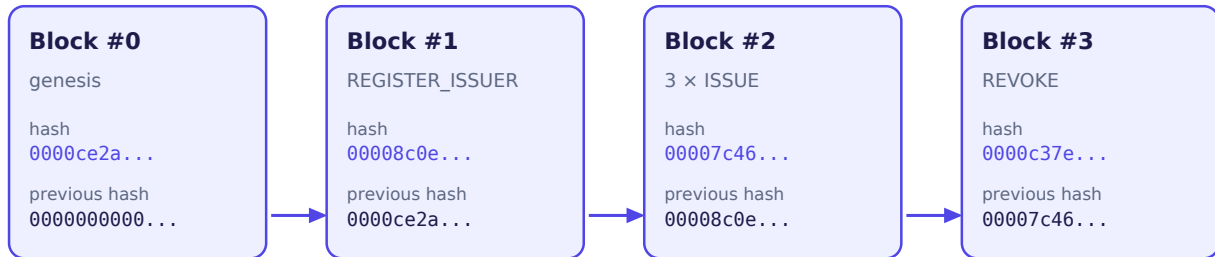


The full record is saved in the database (off-chain); only the hash goes on the blockchain.

Figure 2: Credential issuance flow

4.3 Blockchain structure

Each block stores its index, timestamp, the previous block's hash, the Merkle root of its transactions, the nonce found by mining and its own hash. The first block (genesis) has a previous hash of 64 zeros.



Each block stores the previous block's hash, so changing one block breaks every link after it.

Figure 3: Hash-linked blocks

4.4 Verification flow

Verification runs six checks and produces one of four results. VALID requires every check to pass. PENDING means the credential is authentic but not yet mined. REVOKED means it is authentic but cancelled by the issuer. INVALID means at least one integrity check failed.

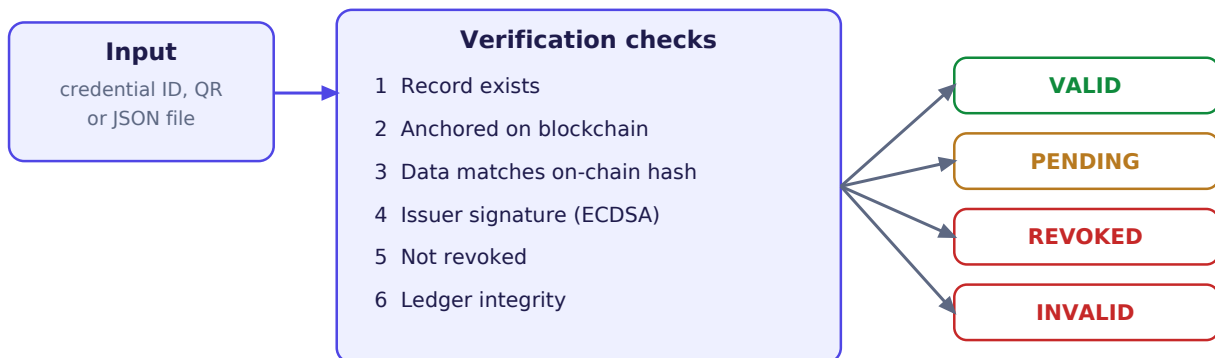


Figure 4: Verification checks and outcomes

4.5 Database design

Personal data lives off-chain in relational tables; the blockchain itself is persisted as one JSON document per block, so the chain survives restarts and can be re-validated from storage at any time.

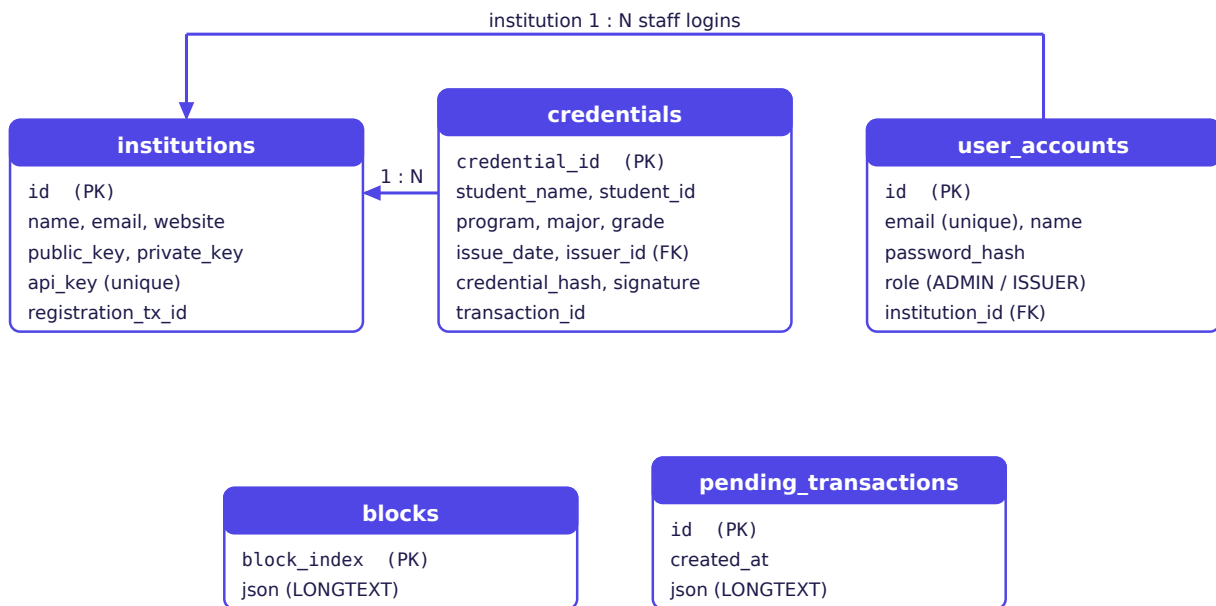


Figure 5: Database tables and relationships

4.6 Transaction types

Type	Purpose	Signed by
REGISTER_ISSUER	Anchors an institution's public key and name on the chain	The institution itself (self-signed)
ISSUE	Records the SHA-256 hash of a new credential	The issuing institution
REVOKE	Cancels a credential with a reason; history is kept	The original issuing institution only

Table 4: Blockchain transaction types

4.7 Module description

Package	Responsibility
crypto	HashUtil (SHA-256) and KeyUtil (ECDSA key generation, signing, verification)
blockchain	Block, Transaction, MerkleTree and the Blockchain service: pending pool, mining, validation, persistence
credential	Issue, revoke, verification (six checks), signed JSON document
institution	Institution registration and key management
auth	User accounts, PBKDF2 password hashing, JWT tokens, roles, logout, change password
web	REST controllers, error handling, attack-lab endpoints
config	CORS settings and demo data seeding

Table 5: Backend packages

5. Implementation

5.1 Technology stack

Layer	Technology	Reason
Language	Java 17+	Strong typing; cryptography built into the JDK
Backend framework	Spring Boot 3.3, Spring Data JPA	REST API, validation and persistence with little boilerplate
Database	H2 / MySQL 8	Zero-setup development; production-grade option
Frontend	React 18 + Vite	Interactive single-page interface with fast builds
PDF & QR	jsPDF, qrcode	Printable certificates generated in the browser
Testing	JUnit 5, Spring MockMvc	Unit and end-to-end API tests

Table 6: Technology stack

5.2 Hashing a credential

All fields are written in a fixed order into one string, so the same data always produces the same hash.

```
public String canonical() {
    return String.join("\n",
        "credentialId=" + clean(credentialId),
        "studentName=" + clean(studentName),
        "grade=" + clean(grade),
        /* ... all other fields in fixed order ... */
        "issuerName=" + clean(issuerName));
}
public String hash() { return HashUtil.sha256(canonical()); }
```

5.3 Proof-of-work mining

```
public void mine() {
    String target = "0".repeat(difficulty);    // difficulty 4 -> "0000"
    nonce = 0;
    hash = calculateHash();
    while (!hash.startsWith(target)) {
        nonce++;
        hash = calculateHash();                // SHA-256 of header + nonce
    }
}
```

5.4 Merkle root

```
while (level.size() > 1) {
    List<String> next = new ArrayList<>();
    for (int i = 0; i < level.size(); i += 2) {
        String left = level.get(i);
        String right = (i + 1 < level.size()) ? level.get(i + 1) : left;
        next.add(HashUtil.sha256(left + right));
    }
    level = next;
}
```

5.5 Digital signatures

```
Signature s = Signature.getInstance("SHA256withECDSA"); // curve secp256r1 (P-256)
s.initSign(privateKey);
s.update(data.getBytes(UTF_8));
return Base64.getEncoder().encodeToString(s.sign());
```

5.6 Chain validation

Validation walks every block and recomputes: the block hash, the proof-of-work target, the link to the previous block, the Merkle root, each transaction ID and each transaction signature, using the issuer public keys found on the chain itself. Every failure is reported with the block number and the reason.

5.7 Authentication

Passwords are hashed with PBKDF2-HMAC-SHA256 using a random 16-byte salt and 210,000 iterations, and compared in constant time. After login the server issues a JWT signed with HMAC-SHA256 containing the user ID, role, institution and an 8-hour expiry. Five failed logins lock the account for five minutes, and the same error is returned for unknown emails and wrong passwords.

5.8 Main REST endpoints

Method & path	Access	Purpose
POST /api/auth/login	Public	Sign in, returns JWT
POST /api/institutions	Admin	Register institution and staff login
POST /api/credentials	Issuer	Issue a credential
POST /api/credentials/{id}/revoke	Issuer	Revoke with reason
GET /api/verify/{id}	Public	Verify by credential ID
POST /api/verify/document	Public	Verify an uploaded JSON document
POST /api/chain/mine	Public	Mine pending transactions
GET /api/chain/validate	Public	Full ledger integrity report
POST /api/demo/*	Admin	Attack simulation and restore

Table 7: Main API endpoints

6. Results and Screenshots

The screenshots below were taken while running the application.

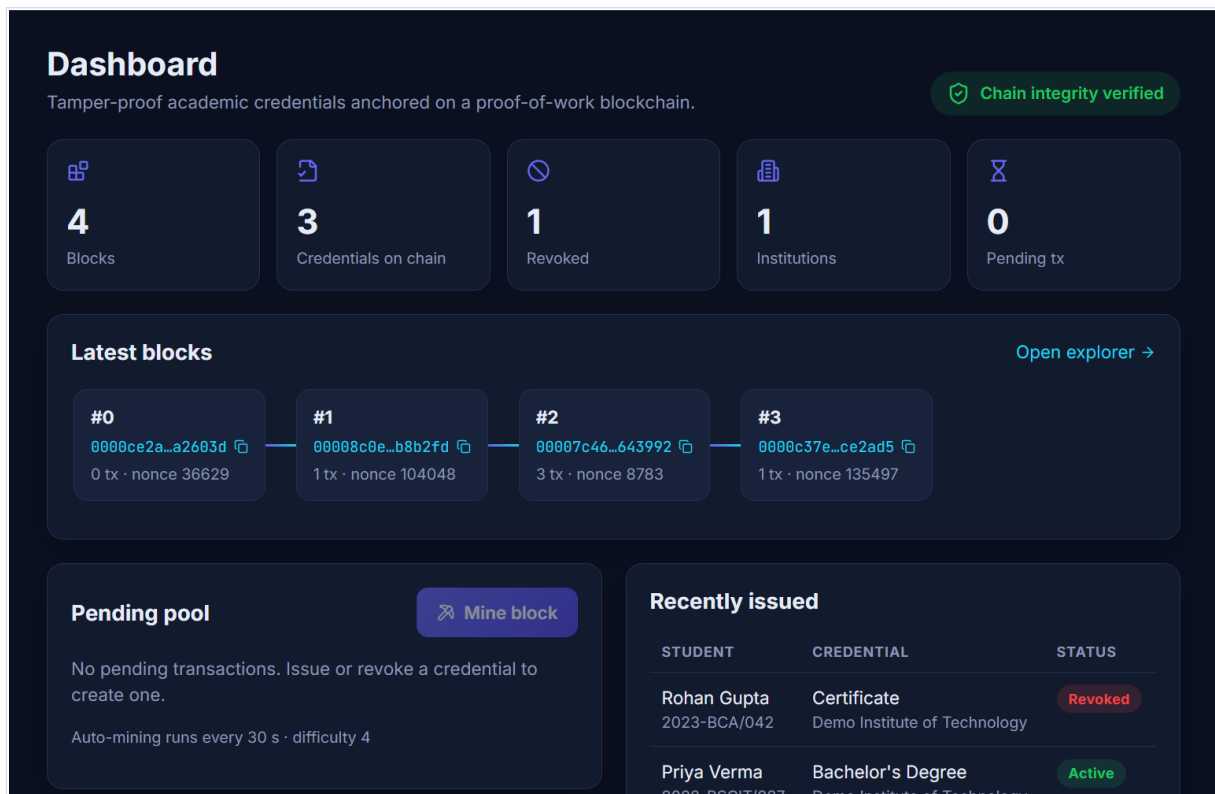


Figure 6: Dashboard with chain statistics and linked blocks

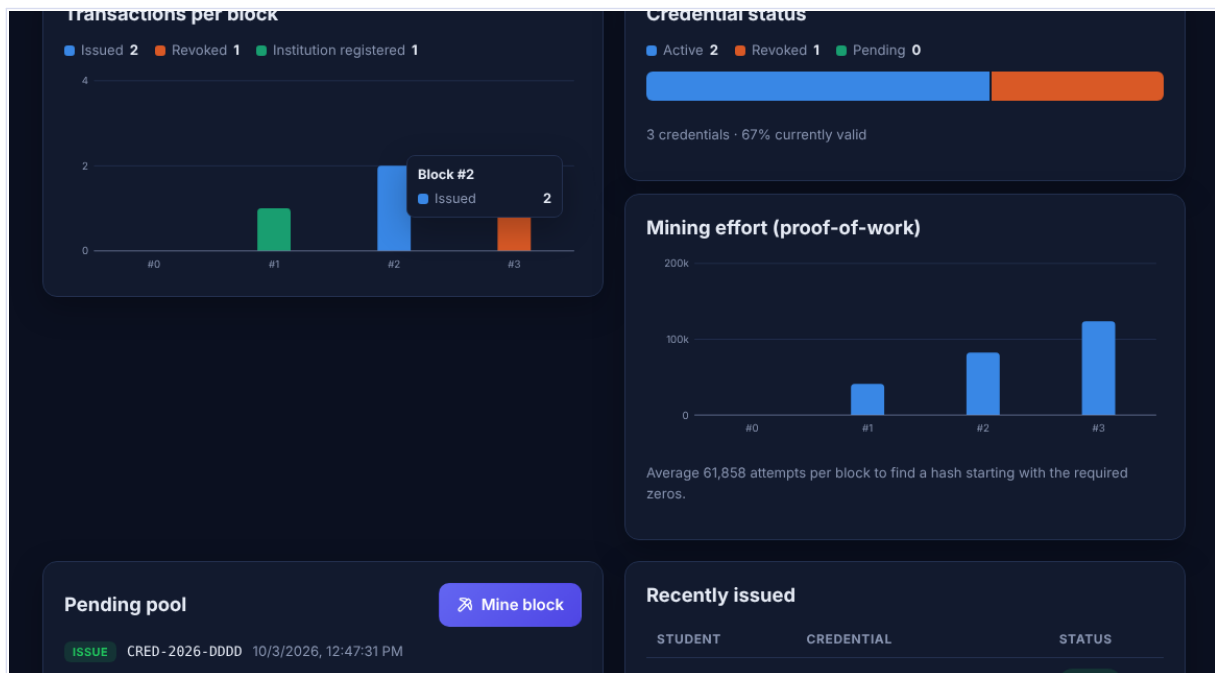


Figure 7: Dashboard charts: transactions per block, credential status and mining effort

Sign in

For college staff and administrators. Verifying a credential needs no login.

Email

registrar@demo-institute.edu

Password

.....

👁

→ Sign in

🔑 Demo accounts

College staff	registrar@demo-institute.edu	demo123
Admin	admin@credchain.local	admin123

Click a row to fill the form. Passwords are stored as salted PBKDF2 hashes, never as plain text.

Figure 8: Sign-in page for staff and administrators

Student name

Student / roll ID

Program

B.Sc. Information Technology

Major / specialisation

Grade / CGPA

Issue date

dd-mm-yyyy

↗ Sign & issue

The record is hashed (SHA-256), signed with your ECDSA private key, and only the hash goes on-chain.

Cecil Baraik

(2025-BSCIT/004A) has successfully completed **B.Sc. Information Technology** with specialisation in **Cyber Security**

GRADE	ISSUED ON	CREDENTIAL ID	BLOCK
9.0 CGPA	2026-10-03	CRED-2026-28AB00EDE6	pending

SHA-256 FINGERPRINT

f8566e501ce3092a...b6c7f0

Scan to verify

Open credential page

Credentials issued by Demo Institute of Technology

ID	STUDENT	CREDENTIAL	BLOCK	STATUS
CRED-2026-28AB00EDE6	Cecil Baraik 2025-BSCIT/004A	Bachelor's Degree B.Sc. Information Technology	—	Pending 🚫 Revoke

Figure 9: Issuer Portal after issuing a credential (status: pending)

Bulk issue

Issue certificates for a whole class from one CSV file · Demo Institute of Technology

Download template

class.csv or click to choose

Columns: credentialType, studentName, studentId, program, major, grade, issueDate (yyyy-mm-dd or dd-mm-yyyy). Max 500 rows.

From Excel: File → Save As → CSV UTF-8 (Comma delimited).

Preview · 5 rows

3 ready

2 with errors (skipped)

Sign & issue 3

ROW	STUDENT	CREDENTIAL	GRADE	ISSUE DATE	CHECK
2	Ananya Singh 2022-BSCIT/031	Bachelor's Degree B.Sc. Information Technology · Cyber Security	8.4 CGPA	2025-06-30	OK
3	Rahul Kumar 2022-BSCIT/032	Bachelor's Degree B.Sc. Information Technology · Cloud Computing	7.9 CGPA	2025-06-30	OK
4	Oraon, Sneha 2024-BCA/015	Certificate Ethical Hacking Workshop	A	2026-02-15	OK
5	Bad Row	Certificate Workshop	B	2099-01-01	studentId is empty; issueDate is in the future
6	Ananya Singh 2022-BSCIT/031	Bachelor's Degree B.Sc. Information Technology · Cyber Security	8.4 CGPA	2025-06-30	duplicate of row 2

Figure 10: Bulk issue from CSV with row-by-row validation

Dashboard

Tamper-proof academic credentials anchored on a proof-of-work blockchain.

Chain integrity verified

5
Blocks

4
Credentials on chain

1
Revoked

1
Institutions

0
Pending tx

Latest blocks

Open explorer →

#0
0000ce2a...a2603d
0 tx · nonce 36629

#1
00008c0e...b8b2fd
1 tx · nonce 104048

#2
00007c46...643992
3 tx · nonce 8783

#3
0000c37e...ce2ad5
1 tx · nonce 135497

#4
0000cd2b...ac70f6
1 tx · nonce 125892

Pending pool

No pending transactions. Issue or revoke a credential to create one.

Auto-mining runs every 30 s · difficulty 4

Mine block

Recently issued

STUDENT	CREDENTIAL	STATUS
Cecil Baraik 2025-BSCIT/004A	Bachelor's Degree Demo Institute of Technology	Active
Rohan Gupta	Certificate	Revoked

Figure 11: After mining: block #4 added and the credential becomes active

CredentialChain – Blockchain-Based Academic Credential System

Page 21

Check any credential against the blockchain in seconds — scan its QR code, enter its ID, or upload the JSON file.

Credential verified
 Authentic credential issued by Demo Institute of Technology.

Verification steps	Credential details
Record exists Found credential CRED-2026-28AB00EDE6	Student Cecil Baraik (2025-BSCIT/004A)
Anchored on blockchain Found in block #4	Credential Bachelor's Degree
Data matches on-chain hash SHA-256 f8566e501ce3... matches the blockchain	Program B.Sc. Information Technology — Cyber Security
Issuer signature (ECDSA) Signed by Demo Institute of Technology (INST-E3F2E75D)	Grade 9.0 CGPA
Not revoked No revocation recorded	Issued 2026-10-03
Ledger integrity All 5 blocks verified	Issuer Demo Institute of Technology (INST-E3F2E75D)
	Hash f8566e501ce309...b6c7f0
	Transaction 94f29a98f3...4f588c
	Block #4 · 03/10/2026, 10:30:37

Figure 12: Verification of a genuine credential – all six checks pass

The file is re-hashed and its signature checked against the issuer's on-chain public key.

Verification failed
 Verification failed — this credential cannot be trusted.

Verification steps	Credential details
Document integrity Document content was edited: recomputed hash 2dd1407b089f... ≠ f8566e501ce3...	Student Cecil Baraik (2025-BSCIT/004A)
Anchored on blockchain Found in block #4	Credential Bachelor's Degree
Data matches on-chain hash Data was altered: 2dd1407b089f... ≠ on-chain f8566e501ce3...	Program B.Sc. Information Technology — Cyber Security
Issuer signature (ECDSA) Signed by Demo Institute of Technology (INST-E3F2E75D)	Grade 10.0 CGPA
Not revoked No revocation recorded	Issued 2026-10-03
Ledger integrity All 5 blocks verified	Issuer Demo Institute of Technology (INST-E3F2E75D)
	Hash 2dd1407b089f2a...f37812
	Transaction 94f29a98f3...4f588c
	Block #4 · 03/10/2026, 10:30:37

Figure 13: Uploaded document with the grade edited – verification fails

By credential ID

Upload credential file

CRED-2026-28AB00EDE6

Verify

Credential revoked

Authentic, but revoked by the issuer: Issued by mistake

Verification steps

Record exists

Found credential CRED-2026-28AB00EDE6

Anchored on blockchain

Found in block #4

Data matches on-chain hash

SHA-256 f8566e501ce3... matches the blockchain

Issuer signature (ECDSA)

Signed by Demo Institute of Technology (INST-E3F2E75D)

Not revoked

Revoked: Issued by mistake

Ledger integrity

All 6 blocks verified

Credential details

Student

Cecil Baraik (2025-BSCIT/004A)

Credential

Bachelor's Degree

Program

B.Sc. Information Technology — Cyber Security

Grade

9.0 CGPA

Issued

2026-10-03

Issuer

Demo Institute of Technology (INST-E3F2E75D)

Hash

f8566e501ce309...b6c7f0

Transaction

94f29a90f3...4f588c

Block

#4 · 03/10/2026, 10:30:37

Figure 14: Revoked credential – authentic but cancelled, with reason

By credential ID

Upload credential file

CRED-2026-0CF629B268

Verify

Verification failed

Verification failed — this credential cannot be trusted.

Verification steps

Record exists

Found credential CRED-2026-0CF629B268

Anchored on blockchain

Found in block #2

Data matches on-chain hash

Data was altered: fea37aac4ee8... ≠ on-chain 9efb64daf33a...

Issuer signature (ECDSA)

Signed by Demo Institute of Technology (INST-E3F2E75D)

Not revoked

No revocation recorded

Ledger integrity

All 6 blocks verified

Credential details

Student

Priya Verma (2022-BSCIT/027)

Credential

Bachelor's Degree

Program

B.Sc. Information Technology — Cloud Computing

Grade

10.0 CGPA (forged)

Issued

2025-06-30

Issuer

Demo Institute of Technology (INST-E3F2E75D)

Hash

fea37aac4ee86e...a51811

Transaction

d017490adf...a2f9aa

Block

#2 · 03/10/2026, 10:22:08

Figure 15: Attack lab: grade changed in the database is detected

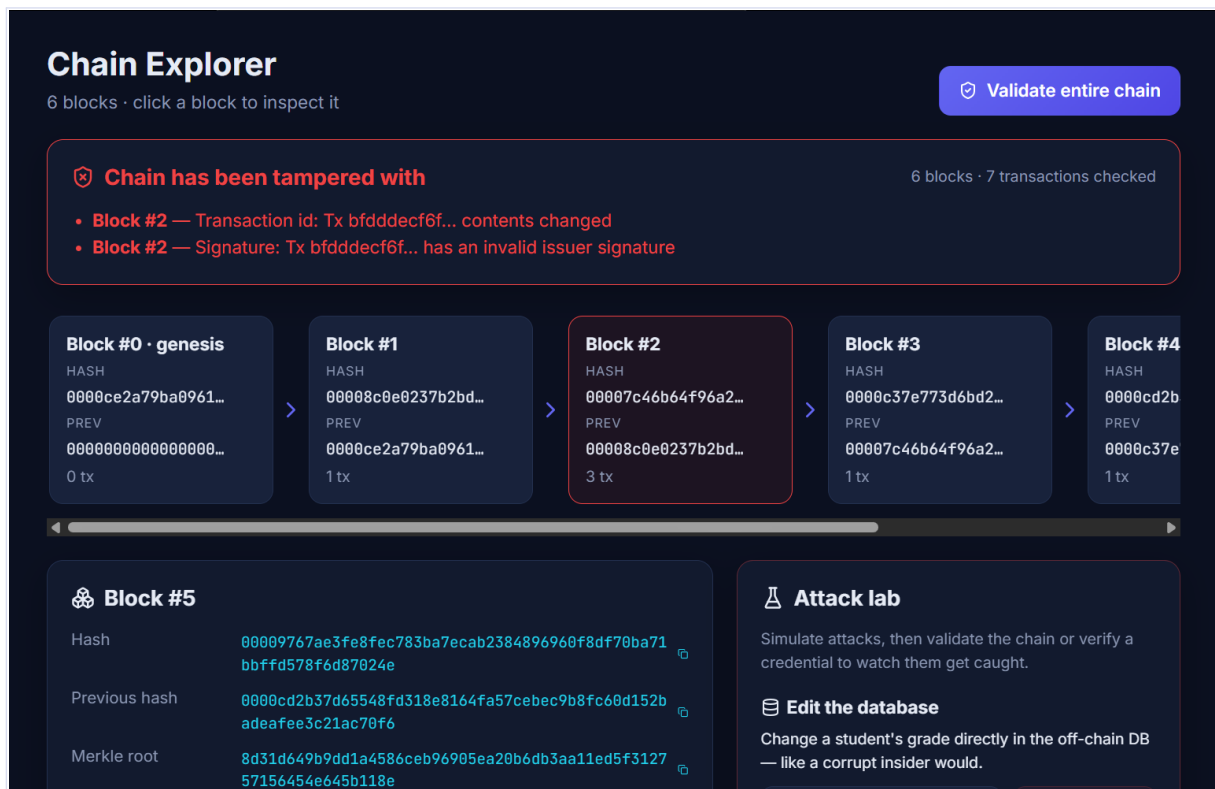


Figure 16: Attack lab: edited block detected by full chain validation

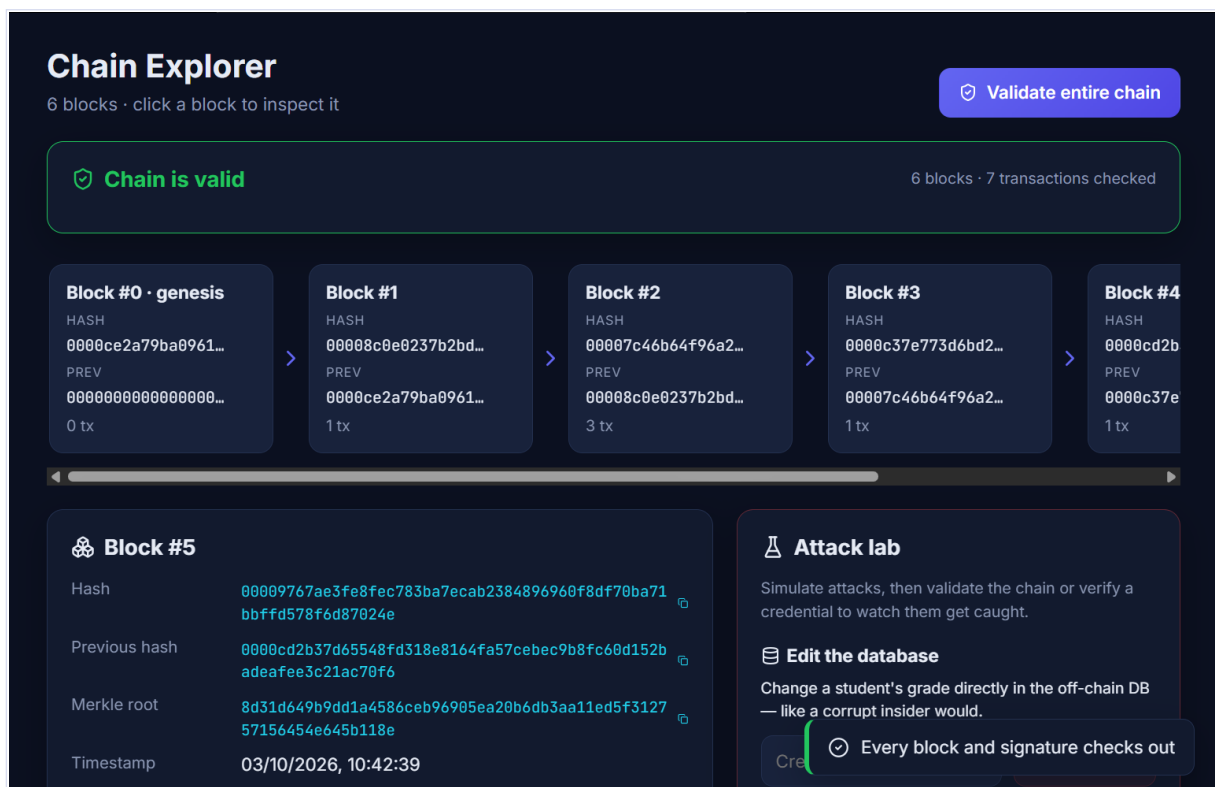


Figure 17: Chain valid again after restore



Figure 18: Printable PDF certificate with QR code (revoked example)

```
Query OK, 5 rows affected (0.12 sec)

mysql> CREATE DATABASE credchain;
Query OK, 1 row affected (0.01 sec)

mysql> USE credchain;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_credchain |
+-----+
| blocks               |
| credentials          |
| institutions         |
| pending_transactions |
| user_accounts       |
+-----+
5 rows in set (0.01 sec)

mysql> SELECT credential_id, student_name, grade FROM credentials;
+-----+-----+-----+
| credential_id | student_name | grade |
+-----+-----+-----+
| CRED-2026-37688336BB | Priya Verma | 9.1 CGPA |
| CRED-2026-3A60A29631 | Rohan Gupta | A+ |
| CRED-2026-D942D94F48 | Aarav Sharma | 8.7 CGPA |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Figure 19: Data stored in MySQL

7. Testing

7.1 Testing approach

Unit tests (JUnit 5) check the building blocks: SHA-256 output, ECDSA sign/verify, mining, Merkle root sensitivity, credential hash sensitivity and transaction tamper detection. An integration test (Spring Boot + MockMvc) runs the whole lifecycle through the REST API. Every user flow was also tested manually in the browser.

7.2 Test cases

ID	Test case	Expected result	Result
TC1	Issue a credential with valid data	Credential created, status PENDING	Pass
TC2	Mine the pending pool	New block with hash starting "0000"; status ACTIVE	Pass
TC3	Verify a genuine credential by ID	VALID, all six checks green	Pass
TC4	Upload a downloaded JSON with grade edited	INVALID – hash and signature mismatch	Pass
TC5	Change grade directly in the database	INVALID – data does not match on-chain hash	Pass
TC6	Edit a transaction inside a mined block	Chain validation reports tampered block	Pass
TC7	Restore after attacks	Chain valid, credential valid again	Pass
TC8	Revoke a credential	REVOKED with the given reason	Pass
TC9	Revoke a credential twice	Rejected – already revoked	Pass
TC10	Another institution tries to revoke	Rejected (403)	Pass
TC11	Issue with a future date	Rejected with clear message	Pass
TC12	Issue without signing in	Rejected (401)	Pass
TC13	Admin tries to issue a credential	Rejected (403) – wrong role	Pass
TC14	Sign in with wrong password	Rejected with generic message	Pass
TC15	Five wrong passwords in a row	Account locked for five minutes	Pass
TC16	Edit role inside a login token	Rejected – invalid signature	Pass
TC17	Use the attack lab without admin login	Rejected (401 / 403)	Pass
TC18	Bulk CSV with missing fields, future dates and duplicates	Bad rows flagged and skipped; valid rows issued	Pass
TC19	Verify an unknown credential ID	NOT FOUND	Pass
TC20	Run on MySQL with the "mysql" profile	Tables created and data stored in MySQL	Pass

Table 8: Test cases

8. Security Analysis

Threat	How CredentialChain handles it
Forged certificate document	Recomputed SHA-256 differs from the on-chain hash; the issuer signature no longer matches.
Insider edits the database	The edited record no longer matches its on-chain fingerprint, so verification fails.
Attacker edits blockchain history	Transaction IDs, signatures, Merkle roots and hash links break; validation reports the block.
Fake issuer key inserted in database	Verification uses public keys anchored on the chain, not the database copy.
Stolen password database	Passwords are salted PBKDF2 hashes with 210,000 iterations.
Password guessing	Account lockout after five failures; same message for wrong email or password.
Token tampering	JWT signature check fails if any claim, such as the role, is changed.
Unauthorised issuing or revoking	Role checks; only the original issuer may revoke its credentials.
Privacy of student data	Only hashes are stored on-chain; personal data stays off-chain.

Table 9: Threats and mitigations

9. Limitations and Future Scope

9.1 Limitations

- The blockchain runs on a single node, so it is tamper-evident but not decentralised.
- Institution private keys are stored by the server so it can sign on their behalf; in production each institution should hold its own key.
- Proof-of-work at difficulty 4 demonstrates the concept but is not meant to resist large-scale computing attacks.
- QR codes on other devices work only when the application is reachable over the network.

9.2 Future scope

- Multiple nodes with a consensus algorithm, or deployment on Ethereum or Hyperledger Fabric.
- Institutions sign in their own browser or hardware wallet so private keys never leave them.
- W3C Decentralized Identifiers (DIDs) for issuers and students.
- Two-factor authentication for staff and password reset by email.
- Email delivery of credentials to students and IPFS storage for documents such as transcripts.

10. Conclusion

CredentialChain shows that the core ideas of blockchain – hash chaining, Merkle trees, proof-of-work and digital signatures – can make academic credentials verifiable in seconds and practically impossible to forge undetected. By storing only fingerprints on-chain, the system protects student privacy while still letting anyone check authenticity without contacting the institution.

The project delivers a complete working application: institutions issue and revoke credentials, students share them as QR-coded PDFs or signed files, and verifiers get a clear, step-by-step result. Simulated attacks on the document, the database and the blockchain itself were all detected, meeting the project's objectives.

References

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
- [2] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," CRYPTO '87, 1987.
- [3] W3C, "Verifiable Credentials Data Model," W3C Recommendation.
- [4] Blockcerts – The Open Standard for Blockchain Credentials, blockcerts.org.

- [5] NIST FIPS 180-4, "Secure Hash Standard (SHS)."
- [6] NIST FIPS 186, "Digital Signature Standard (DSS)."
- [7] IETF RFC 7519, "JSON Web Token (JWT)," 2015.
- [8] IETF RFC 8018, "PKCS #5: Password-Based Cryptography Specification (PBKDF2)," 2017.
- [9] OWASP, "Password Storage Cheat Sheet."
- [10] Spring Boot Reference Documentation, spring.io; React Documentation, react.dev.